

Rapport PokéHack

Jean-Simon Cyr (Frontend), Émile Valade (Backend)

Séparation des tâches

Setup & Architecture

- Création du dépôt Git (Émile)
- Configuration de Maven + JavaFX (Jean-Simon)
- Configuration de JDBC (Jean-Simon)
- Diagramme de relation d'entité (Émile)
- Création de la base de données (Émile)
- Création des tables (Émile)
- Squelette MVC (Émile)

Backend & Service API

- Création des POJO de chaque classe (Émile)
- Création des DAO de chaque classe (Émile)
- Développement du `PokemonApiService` (Émile)
- Tests unitaires du service (Jean-Simon)

Vue JavaFX

- Développement complet de la vue (Jean-Simon)

Controller & Multithreading

- Développement du contrôleur (Jean-Simon)

Design & CSS

- CSS des 18 types de Pokémon (Jean-Simon)
- Icônes (Jean-Simon)
- Layout responsive (Jean-Simon)

Qualité du code

- Revue des *pull requests* (Émile & Jean-Simon)
- Tests du code (Émile & Jean-Simon)

Présentation & Git

- Rédaction du README (Émile & Jean-Simon)
 - Rédaction du rapport (Émile & Jean-Simon)
-

Choix techniques

L'API PokeAPI retourne une grande quantité de données, à la fois simples (`String`, `int`, `float`, etc.) et complexes (JSON). Afin de conserver le plus d'informations possible tout en respectant les principes de normalisation d'une base de données relationnelle, nous avons séparé les différents objets et leurs valeurs dans plusieurs tables.

En suivant cette logique, toute information que nous souhaitions conserver et qui pouvait être partagée entre plusieurs Pokémon a été placée dans une table distincte avec une table de jonction.

C'est pourquoi nous avons créé des tables de jonction entre `pokemon` et `move`, `pokemon` et `ability`, ainsi qu'entre `pokemon` et `type`. Toutes les autres relations peuvent être représentées à l'aide de clés étrangères.

Difficultés rencontrées

Émile :

À plusieurs reprises durant le développement du backend, j'ai dû revenir sur la base de données afin de modifier le type de certaines colonnes, puis répercuter ces changements dans les modèles et les DAO. Ce problème s'est accentué lorsque j'ai commencé à développer le `PokemonApiService`, puisque l'API ne retournait pas toujours les données dans le format attendu ou ne les structurait pas comme je l'avais prévu.

Avec le recul, je pense qu'il aurait été préférable de prendre davantage de temps pour bien analyser les données retournées par l'API avant de concevoir la base de données. Cela aurait probablement permis d'éviter plusieurs modifications en cours de développement.

Une autre difficulté a été de ne pas réussir à utiliser efficacement les `enum` que j'avais créés. À aucun moment ils ne m'ont semblé offrir un réel avantage par rapport au stockage direct des chaînes de caractères ou à l'utilisation de tables semi-statiques dans la base de données.

Jean-Simon :

Ma principale difficulté a été de construire une interface JavaFX complète avec pas beaucoup d'expérience sur ce framework. J'ai dû apprendre en avançant, surtout pour comprendre le rôle des différents conteneurs (`GridPane`, `VBox`, `HBox`, `StackPane`) et la façon dont ils influencent la position et la taille des éléments.

Au début, certains choix de noms et de structure n'étaient pas assez clairs, ce qui rendait les ajustements visuels plus difficiles. J'ai aussi dû mieux comprendre quelles modifications devaient être faites en JavaFX et lesquelles devaient plutôt être placées dans le CSS. Par exemple, certaines tailles, couleurs et bordures étaient plus faciles à gérer en CSS, tandis que la structure générale de l'interface devait rester dans les classes JavaFX.

Le responsive design a été une autre difficulté importante. Les marges fixes donnaient un résultat correct à une taille précise, mais l'interface devenait moins stable lorsque la fenêtre changeait de taille. J'ai finalement utilisé une approche basée sur une taille de référence, avec un `Group`, un `StackPane` et un facteur de scale pour conserver les proportions de l'interface.

Mes connaissances en interface homme-machine (HMI), acquises comme technicien en automatisation, m'ont aidé à visualiser l'interface sous forme de zones et de grilles. Cela m'a donné une base pour organiser l'écran, même si l'adaptation à JavaFX a demandé plusieurs essais.

Ajouts avec plus de temps

Émile :

Avec davantage de temps, je repenserais légèrement la structure de la base de données afin d'améliorer certaines relations. J'intégrerais également plus proprement les données statiques, comme les `type` ou les `move_damage_class`, sous forme d'`enum` lorsque cela est pertinent.

Jean-Simon :

Avec plus de temps, j'améliorerais la gestion des erreurs et les validations d'entrée afin de donner plus de feedback à l'utilisateur. Par exemple, l'application pourrait afficher des messages plus précis lorsqu'un identifiant est invalide, lorsque la connexion à l'API échoue ou lorsqu'une donnée n'est pas disponible.

J'aimerais aussi améliorer davantage le design en m'inspirant du Pokédex officiel. Une idée serait d'ajouter des flèches de navigation pour passer rapidement au Pokémon précédent ou suivant selon son identifiant. J'aurais aussi aimé pousser plus loin les animations, les transitions entre les vues et l'affichage des chaînes d'évolution.

Finalement, je prendrais plus de temps pour étudier les bonnes pratiques JavaFX afin de mieux séparer la structure de l'interface, le style CSS et la logique du contrôleur. Cela rendrait le code plus facile à maintenir et à modifier.